

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **Hashini S / 192524035** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Hashini S

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem

- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Scenario: AI-Powered Drones for Emergency Medicine Delivery

Introduction

In a flood-affected region, an AI-powered drone must deliver emergency medicines as quickly and safely as possible. The environment is dynamic and uncertain because roads may suddenly become blocked due to floods, weather conditions change frequently, and different terrains increase or decrease travel cost. The drone has only partial real-time information, a heuristic estimate

(straight-line distance to the destination), and variable movement costs depending on terrain and weather. Therefore, selecting an efficient search algorithm is essential for reducing delivery time and ensuring safety.

i. Greedy Best-First Search (GBFS) in this Scenario

Working

Greedy Best-First Search selects the path that appears closest to the goal based only on the heuristic function.

Evaluation Function:

$$\begin{bmatrix} f(n) = h(n) \end{bmatrix}$$

Where:

- **$f(n)$** = Estimated value of a node
- **$h(n)$** = Heuristic estimate (straight-line distance to the destination)

The algorithm always expands the node with the smallest heuristic value, assuming it is the fastest route.

Example

If the drone has two possible routes:

- Route A → Straight-line distance = 10 km, but flooded and risky.
- Route B → Straight-line distance = 15 km, but safe and clear.

Greedy Best-First Search chooses Route A because it looks closer to the destination, even though it may become blocked later.

Strengths

- Finds a route very quickly.
- Requires less computation.
- Suitable when immediate decisions are required.
- Works well if the heuristic is highly accurate.

Weaknesses

- Does not consider the actual travel cost.
- May choose dangerous or blocked routes.
- Cannot guarantee the shortest or safest path.
- Performance decreases when the environment changes frequently.

Conclusion

GBFS is fast, but it is not reliable in uncertain environments because it ignores travel cost and possible risks.

*ii. A Search in the Same Scenario**

Working

A* Search considers both:

1. The actual cost already travelled.
2. The estimated remaining distance.

Evaluation Function:

$$\begin{bmatrix} f(n) = g(n) + h(n) \end{bmatrix}$$

Where:

- **$g(n)$** = Actual cost from the starting point to node n
- **$h(n)$** = Estimated cost from node n to the goal
- **$f(n)$** = Total estimated cost

Unlike Greedy Search, A* balances both present and future costs.

Example

Suppose:

Route A

- Travel cost = 25
- Heuristic = 10

$$\begin{aligned} &[\\ f(A) &= 25 + 10 = 35 \\ &] \end{aligned}$$

Route B

- Travel cost = 12
- Heuristic = 15

$$\begin{aligned} &[\\ f(B) &= 12 + 15 = 27 \\ &] \end{aligned}$$

Although Route A is closer to the goal, A* selects Route B because its total estimated cost is lower and safer.

Advantages

- Produces the optimal path when the heuristic is admissible.
- Considers both distance and movement cost.
- Avoids expensive or risky paths.
- Better suited for real-world navigation.

Disadvantages

- Requires more memory.
- Performs more calculations.
- Slightly slower than Greedy Search.

Conclusion

A* provides a better balance between speed and accuracy, making it suitable for emergency drone navigation.

iii. Impact of Heuristic Accuracy on Both Algorithms

The heuristic function estimates the remaining distance to the destination. Its accuracy directly affects the performance of both algorithms.

For Greedy Best-First Search

- Highly dependent on the heuristic.
- Accurate heuristic → Fast and efficient.
- Poor heuristic → May follow completely wrong routes.
- Can become trapped in dead ends or blocked paths.

For A*

- Uses both heuristic and actual travel cost.
- Even if the heuristic is slightly inaccurate, the actual cost helps correct the search.
- With an admissible heuristic, A* guarantees the optimal solution.
- More reliable under uncertain conditions.

Comparison

Greedy Best-First Search	A* Search
Depends only on heuristic	Uses heuristic and actual cost
Sensitive to heuristic errors	Less affected by heuristic errors
May choose wrong path	Produces better decisions
Faster	Slightly slower but more accurate

Conclusion

A* is less dependent on heuristic accuracy and therefore performs more consistently in complex environments.

iv. Effect of Dynamic Changes (Blocked Paths)

Flood conditions change continuously. Roads may become blocked, and weather may increase travel costs.

Greedy Best-First Search

- Continues toward the nearest goal.

- If the selected path becomes blocked, it must search again.
- Wastes time exploring unsuitable paths.
- Delivery may be delayed.

A Search*

- Detects increased travel costs.
- Recalculates the total cost using $g(n)+h(n)$.
- Selects an alternative route with lower total cost.
- Better adapts to changing conditions.

Comparison

Dynamic Situation	Greedy Best-First Search	A* Search
Road blocked	Often backtracks	Finds alternate route efficiently
Weather changes	Ignores cost changes	Updates route based on new cost
Safety	Lower	Higher
Reliability	Moderate	High

Conclusion

A* handles dynamic environments much better because it continuously evaluates both travel cost and estimated distance.

v. Recommended Algorithm

Recommended Algorithm: A Search*

Justification

A* Search is the most suitable algorithm for emergency medicine delivery because:

- It considers both actual travel cost and estimated remaining distance.
- It finds the safest and shortest path.
- It adapts better to blocked roads and changing weather.
- It minimizes delivery delay while avoiding risky routes.

- It guarantees the optimal path when the heuristic is admissible.
- Although it requires more computation than Greedy Search, modern drones can process A* efficiently in real time.

Final Comparison

Feature	Greedy Best-First Search	A* Search
Speed	Very Fast	Fast
Uses Actual Cost	No	Yes
Uses Heuristic	Yes	Yes
Finds Optimal Path	No	Yes
Handles Dynamic Changes	Poor	Good
Safety	Moderate	High
Best for Emergency Delivery	No	Yes

2. Scenario: AI-Based Smart City Traffic Signal Optimization

Introduction

A smart city uses an Artificial Intelligence (AI) system to optimize traffic signal timings across hundreds of road intersections. The primary objectives are to minimize total waiting time, reduce fuel consumption, and decrease traffic congestion. Since traffic conditions continuously change due to accidents, weather, and varying vehicle flow, the AI system must frequently adjust signal timings. The search space is extremely large, making it difficult for traditional search algorithms to find the best solution efficiently. Therefore, optimization techniques such as Hill Climbing, Simulated Annealing, and Genetic Algorithms are more suitable.

Formulation as an Optimization Problem and Why Traditional Search is Inefficient

Optimization Problem

The traffic signal optimization problem can be formulated as follows:

Objective Function

Minimize:

- Total vehicle waiting time
- Fuel consumption
- Traffic congestion

Constraints

- Traffic volume changes continuously.
- Signal timings must remain within legal limits.
- Emergency vehicles should receive priority.
- Pedestrian crossing time must be maintained.
- Adjacent intersections should remain synchronized.

Decision Variables

- Green signal duration
- Red signal duration
- Yellow signal duration
- Signal sequence at each intersection

The AI system searches for the combination of signal timings that produces the lowest traffic delay and fuel consumption.

Why Traditional Search is Inefficient

Traditional search algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS) become inefficient because:

- Hundreds of intersections create millions of possible signal combinations.
- Traffic conditions change every few seconds.
- Searching every possible solution requires enormous computation time.
- Real-time decisions are impossible using exhaustive search.
- Memory requirements become extremely high.

Therefore, optimization algorithms are preferred because they can quickly find near-optimal solutions without exploring the entire search space.

i. Hill Climbing in this Scenario

Working

Hill Climbing is a local search algorithm that starts with an initial solution and repeatedly moves to a neighboring solution if it improves the objective function.

Steps

1. Start with current traffic signal timings.
2. Slightly modify one or more signal timings.
3. Evaluate whether traffic waiting time decreases.
4. If the new solution is better, move to it.
5. Repeat until no better neighboring solution exists.

Example

Current Signal Timing:

- Green = 30 seconds

New Timing:

- Green = 35 seconds

If increasing the green signal reduces congestion, Hill Climbing accepts the new timing.

This process continues until no neighboring timing improves traffic flow.

Advantages

- Simple to implement.
- Requires very little memory.
- Produces quick improvements.
- Suitable for real-time local optimization.

Limitations

- May stop before reaching the global optimum.

- Easily trapped in local maxima.
- Cannot effectively handle large dynamic environments.
- Small changes may not always improve traffic flow.

ii. Local Maxima, Plateaus, and Ridges (Real-World Interpretation)

1. Local Maximum

A local maximum is a solution that appears to be the best among nearby solutions but is not the best overall.

Traffic Example

The AI adjusts one intersection and reduces waiting time.

However, changing several nearby intersections together would produce much lower congestion.

Since Hill Climbing only examines nearby improvements, it stops early.

Result: Better city-wide traffic flow is never discovered.

2. Plateau

A plateau is a flat region where many neighboring solutions have the same value.

Traffic Example

Changing the green signal from:

- 30 seconds
- 31 seconds
- 32 seconds

produces almost identical waiting times.

The algorithm cannot determine which direction is better.

Result: Search becomes slow or completely stops.

3. Ridge

A ridge is a narrow path where improvement requires multiple coordinated changes.

Traffic Example

Improving traffic requires simultaneously changing signals at several intersections.

Changing only one signal does not improve performance.

Hill Climbing changes only one solution at a time and therefore cannot climb the ridge.

Result: Better traffic coordination is missed.

Summary

Problem	Meaning	Traffic Example
Local Maximum	Best nearby solution but not global best	One intersection optimized while city traffic remains poor
Plateau	Flat search space	Many signal timings produce identical traffic flow
Ridge	Improvement needs multiple coordinated changes	Several intersections must be adjusted together

iii. How Simulated Annealing and Genetic Algorithms Overcome These Limitations

A. Simulated Annealing

Working

Simulated Annealing is an improvement over Hill Climbing.

It sometimes accepts worse solutions during the early search to escape local maxima.

As time passes, the probability of accepting worse solutions decreases.

Advantages

- Escapes local maxima.
- Can cross plateaus.
- Explores larger search areas.
- Produces better global solutions.

Traffic Example

Suppose increasing waiting time slightly at one junction allows smoother traffic flow across the city.

Hill Climbing rejects this move.

Simulated Annealing temporarily accepts it, eventually finding a much better overall solution.

B. Genetic Algorithm (GA)

Working

Genetic Algorithm is inspired by biological evolution.

Each possible traffic signal configuration is treated as a chromosome.

The algorithm repeatedly performs:

- Selection
- Crossover
- Mutation

to generate better traffic signal plans.

Advantages

- Searches many solutions simultaneously.
- Escapes local maxima.
- Handles very large search spaces.
- Works well in dynamic environments.
- Finds high-quality solutions.

Traffic Example

Suppose there are 500 intersections.

Instead of improving one solution, the Genetic Algorithm evaluates many different traffic signal plans simultaneously and combines the best features.

This leads to better city-wide optimization.

Comparison

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes Local Maximum	No	Yes	Yes
Handles Plateau	Poor	Good	Excellent
Handles Ridge	Poor	Good	Excellent
Explores Many Solutions	No	Limited	Yes
Suitable for Large Cities	Moderate	Good	Excellent

iv. Recommended Approach

Recommended Algorithm: Genetic Algorithm

Justification

The Genetic Algorithm is the most suitable approach for smart city traffic optimization because:

- It explores multiple traffic signal plans simultaneously.

- It avoids getting trapped in local maxima.
- It efficiently searches extremely large solution spaces.
- It adapts well to changing traffic conditions.
- It scales easily to hundreds of intersections.
- It provides high-quality solutions for complex optimization problems.

Although Simulated Annealing performs better than Hill Climbing, it explores only one solution at a time.

The Genetic Algorithm maintains an entire population of solutions, making it more suitable for large smart city traffic systems.

Final Comparison

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Speed	Very Fast	Fast	Moderate
Escapes Local Maxima	No	Yes	Yes
Handles Dynamic Traffic	Moderate	Good	Excellent
Scalability	Moderate	Good	Excellent
Finds Near-Optimal Solution	Sometimes	Often	Very High
Best for Smart City Traffic	No	Good	Yes

3. Scenario: Autonomous Mars Rover Using Online Search

Introduction

An autonomous rover is deployed on Mars to explore unknown terrain and collect valuable rock and soil samples. The rover must navigate safely while avoiding hazards such as craters, rocks, and steep slopes. Since it has limited sensing capability and no complete map of the environment, it must make decisions based

on the information it gathers during exploration. Therefore, this is a real-time decision-making problem, making Online Search the most suitable approach.

i. Why This Problem Requires an Online Search Agent Instead of Offline Search

Offline Search

Offline search assumes that:

- The complete environment is already known.
- The entire map is available before planning.
- The agent computes the complete path first and then starts moving.

Why Offline Search is Not Suitable

On Mars:

- The rover does not know the complete terrain.
- New obstacles are discovered while moving.
- The environment is uncertain.
- A previously safe path may become dangerous.

Therefore, offline search cannot produce a reliable plan.

Online Search

Online search allows the rover to:

- Sense the environment while moving.
- Learn new information continuously.
- Update its decisions based on newly discovered obstacles.
- Change its route whenever necessary.

Example

Initially, the rover plans to move straight.

While moving, it detects a large crater.

Instead of continuing, it immediately changes direction and searches for another safe route.

ii. Challenges of Partial Observability and Dynamic Environment

1. Partial Observability

The rover cannot observe the entire environment.

Its sensors detect only nearby objects.

Challenges

- Hidden obstacles.
- Unknown terrain.
- Limited camera and sensor range.
- Incomplete information.

Example

A large rock may be hidden behind a hill.

The rover discovers it only after moving closer.

2. Dynamic Environment

Although Mars changes slowly, new information continuously becomes available.

Examples include:

- Dust storms reducing visibility.
- Loose soil causing wheel slippage.
- Unexpected rocks.
- Sensor uncertainty.

Challenges

- Previously planned paths become unsafe.
- Continuous replanning is required.
- Decisions must be made quickly.

Summary Table

Challenge	Real-World Example
Partial Observability	Hidden rocks beyond sensor range

Dynamic Environment	Dust storm reduces visibility
Uncertainty	Unknown terrain conditions
Limited Sensors	Rover sees only nearby area

iii. How the Rover Builds and Updates Its Internal Model

The rover maintains an internal model of the environment.

Initially:

- Map is almost empty.
- Only the starting position is known.

As the rover moves:

1. Sensors collect new information.
2. Hazards are identified.
3. Safe paths are recorded.
4. Sample locations are stored.
5. The internal map is updated.

Example

Initially:

Unknown Map

↓

Moves Forward

↓

Detects Rock

↓

Marks Rock as Obstacle

↓

Chooses New Path



Updates Map

This process continues throughout the mission.

Advantages

- Improves navigation.
- Avoids revisiting dangerous areas.
- Learns safer routes.
- Increases mission success.

iv. Comparison of Online Search with Uninformed and Informed Search

Feature	Uninformed Search	Informed Search	Online Search
Complete Map Required	Yes	Yes	No
Uses Heuristic	No	Yes	Can Use Heuristic
Learns During Search	No	No	Yes
Adapts to New Obstacles	No	Limited	Yes
Suitable for Unknown Mars Terrain	No	Partially	Yes

Explanation

Uninformed Search

Examples:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)

These require complete knowledge and cannot adapt during execution.

Informed Search

Examples:

- A*
- Greedy Best-First Search

These use heuristic information but still assume the environment is mostly known.

Online Search

The rover:

- Observes.
- Learns.
- Updates.
- Replans continuously.

This makes it ideal for Mars exploration.

v. Recommended Approach

Recommended Algorithm: Online Search Agent

Justification

Online Search is the best approach because it:

- Works without a complete map.
- Continuously updates knowledge.
- Handles uncertainty effectively.
- Adapts to newly discovered hazards.
- Makes safe real-time decisions.
- Supports autonomous exploration.

Final Comparison

Feature	Offline Search	Online Search
Unknown Environment	Poor	Excellent
Real-Time Decisions	No	Yes
Adaptability	Low	High
Safety	Moderate	High
Learning Ability	No	Yes
Best for Mars Rover	No	Yes

4. Scenario: AI-Based University Exam Timetable Generation Using Constraint Satisfaction Problem (CSP)

Introduction

A university is developing an Artificial Intelligence (AI) system to automatically generate an exam timetable. The system must satisfy several important constraints such as ensuring that no student has overlapping exams, managing a limited number of exam halls, scheduling prerequisite subjects before others, considering faculty availability, and providing special accommodations for some students. As the number of students and courses increases, the scheduling process becomes highly complex. Therefore, this problem is best modeled as a Constraint Satisfaction Problem (CSP), where the goal is to find a timetable that satisfies all constraints.

Formulating the Problem as a Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) is a problem where values are assigned to variables while satisfying a set of constraints.

A CSP consists of three main components:

- **Variables**
- **Domains**
- **Constraints**

The objective is to assign suitable values to all variables without violating any constraints.

i. Clearly Define Variables, Domains, and Constraints

1. Variables

Variables represent the exams that need to be scheduled.

Examples:

- Mathematics Exam
- Physics Exam
- Chemistry Exam
- Computer Science Exam
- English Exam

Each exam is considered as one variable whose time and hall must be decided.

2. Domains

The domain is the set of possible values that can be assigned to each variable.

For each exam, the domain includes:

- Available exam dates
- Available time slots
- Available exam halls

Example

Mathematics Exam

Possible Time Slots:

- Monday Morning
- Monday Afternoon
- Tuesday Morning
- Tuesday Afternoon

Possible Halls:

- Hall A
- Hall B
- Hall C

Thus, each exam can be assigned any valid combination of time slot and exam hall.

3. Constraints

The timetable must satisfy all the following constraints.

a) Student Constraint

A student cannot have two exams at the same time.

Example

If a student is enrolled in both Mathematics and Physics, those exams must be scheduled at different times.

b) Exam Hall Constraint

The number of students assigned to an exam hall must not exceed its seating capacity.

Example

Hall A can accommodate only 100 students.

If 150 students are taking Mathematics, another hall must also be assigned.

c) Subject Order Constraint

Certain subjects must be conducted before others.

Example

Programming Fundamentals must be scheduled before Data Structures

d) Faculty Availability Constraint

The assigned faculty member must be available during the examination.

Example

If the Physics faculty is unavailable on Tuesday Morning, the Physics exam cannot be scheduled during that time.

e) Special Accommodation Constraint

Students requiring extra time or special seating arrangements must be provided with appropriate facilities.

Example

Students with disabilities may require a separate examination room and additional writing time.

Summary Table

Component	Explanation	Example
Variables	Exams to be scheduled	Mathematics, Physics
Domains	Possible dates, time slots and halls	Monday Morning, Hall A
Constraints	Rules that must be satisfied	No overlapping exams, hall capacity, faculty availability

ii. Explain How Backtracking Search Works in This Scenario

Working

Backtracking is a systematic search algorithm used to solve CSPs.

It assigns values one by one and checks whether the assignment satisfies all constraints.

If a conflict occurs, the algorithm returns to the previous step and tries another possible value.

Steps

1. Select an exam.
2. Assign an available time slot and exam hall.
3. Check whether all constraints are satisfied.
4. If valid, schedule the next exam.
5. If any constraint is violated, backtrack and choose another available time slot.
6. Continue until all exams are successfully scheduled.

Example

Assign:

Mathematics → Monday Morning ✓

Physics → Monday Morning ✗

Both exams are taken by the same student.

Conflict occurs.

Backtracking changes:

Physics → Tuesday Morning ✓

Now all constraints are satisfied.

The algorithm continues scheduling the remaining exams.

Advantages

- Simple and systematic.
- Guarantees finding a valid timetable if one exists.
- Suitable for solving CSP problems.

Limitations

- Large universities require many backtracking operations.
- Computation time increases rapidly as the number of exams grows.
- Not efficient alone for very large scheduling problems.

iii. Explain How Constraint Propagation (Forward Checking) Improves Efficiency

Working

Constraint Propagation is a technique that reduces the search space by eliminating impossible choices before conflicts occur.

One common method is Forward Checking.

Whenever an exam is assigned a time slot, Forward Checking immediately removes conflicting values from the remaining exams.

This prevents future conflicts and reduces unnecessary searching.

Example

Suppose:

Mathematics $\rightarrow$ Monday Morning

Immediately,

All exams taken by the same students cannot be scheduled on Monday Morning.

Therefore,

Monday Morning is removed from the domains of those exams.

This allows the algorithm to avoid invalid assignments before they happen.

Advantages of Forward Checking

- Detects conflicts early.
- Reduces unnecessary backtracking.
- Decreases computation time.
- Improves scheduling efficiency.
- Makes CSP solving much faster.

Comparison

Backtracking Only	Backtracking + Forward Checking
Detects conflicts later	Detects conflicts early
More backtracking	Less backtracking
Larger search space	Smaller search space
Slower	Faster

iv. Analyze Real-World Challenges and Justify the Best Strategy for Scalability

Real-World Challenges

Large universities face several practical challenges:

- Thousands of students.
- Hundreds of courses.
- Limited exam halls.

- Faculty availability conflicts.
- Special accommodation requirements.
- Last-minute timetable modifications.
- Increasing computational complexity as the university grows.

These factors make timetable generation a difficult optimization problem.

Best Strategy for Scalability

Recommended Approach: Backtracking + Forward Checking

This combination provides the best balance between correctness and efficiency.

Justification

- Backtracking guarantees a valid timetable if one exists.
- Forward Checking removes impossible assignments early.
- Reduces unnecessary search operations.
- Handles large numbers of students and courses efficiently.
- Easily adapts when constraints change.
- Produces high-quality timetables in less computation time.

Final Comparison

Feature	Backtracking	Backtracking + Forward Checking
Guarantees Valid Solution	Yes	Yes
Speed	Moderate	Fast
Backtracking Operations	High	Low
Search Space	Large	Reduced
Scalability	Moderate	High
Suitable for Large Universities	Moderate	Excellent

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

Introduction

A gaming company is developing an Artificial Intelligence (AI) opponent for a real-time strategy game. The AI must compete against human players by making intelligent decisions within a very short time. During gameplay, the AI has many possible moves at every turn, creating an extremely large game tree. Searching the complete game tree is computationally expensive. Therefore, algorithms such as Minimax and Alpha-Beta Pruning are used to make efficient and intelligent decisions while maintaining good game performance.

i. Explain How the Minimax Algorithm Models This Problem

Minimax Algorithm

The Minimax algorithm is a decision-making algorithm used in two-player competitive games such as Chess, Tic-Tac-Toe, Checkers, and Strategy Games.

It assumes:

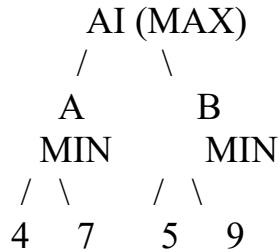
- One player is the MAX player (AI) whose objective is to maximize the score.
- The other player is the MIN player (Human Opponent) whose objective is to minimize the AI's score.

The AI predicts future moves of both players and selects the move that gives the best possible outcome assuming the opponent also plays optimally.

Working of Minimax

1. Generate all possible moves.
2. Construct the game tree.
3. Evaluate the leaf nodes.
4. MAX chooses the highest value.
5. MIN chooses the lowest value.
6. Continue until the best move is selected.

Example



MIN chooses:

- $A \rightarrow 4$
- $B \rightarrow 5$

MAX chooses:

Maximum(4,5)=5

Therefore, the AI selects Move B.

Advantages

- Produces optimal decisions.
- Predicts opponent moves.
- Suitable for competitive games.
- Easy to understand and implement.

Limitations

- Requires searching many game states.
- Computational cost increases rapidly.
- Difficult for real-time games with huge search trees.

ii. Analyze the Computational Challenges of Large Game Trees

Real-time strategy games generate an enormous number of possible moves.

Suppose:

- Each position has 20 possible moves.
- Search depth = 6 levels.

Number of nodes:

[
 $20^6 = 64,000,000$
]

The AI would need to evaluate 64 million game states.

Major Challenges

1. Large Search Space

Every move creates many new possibilities.

The number of game states grows exponentially.

2. Time Constraints

Real-time games require decisions within milliseconds.

Searching the complete tree is too slow.

3. Memory Consumption

Storing millions of game states requires significant memory.

4. Dynamic Gameplay

Human players constantly change strategies.

The AI must repeatedly rebuild and evaluate the game tree.

Conclusion

Without optimization, the Minimax algorithm becomes impractical for large real-time strategy games.

iii. Explain How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization technique for the Minimax algorithm.

It reduces the number of nodes evaluated without affecting the final decision.

It removes branches that cannot influence the final result.

Working

Two values are maintained:

Alpha (α)

The best value that the MAX player can guarantee.

Beta (β)

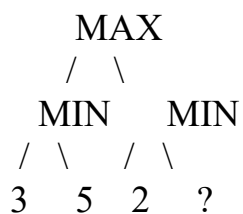
The best value that the MIN player can guarantee.

Whenever:

$$\begin{bmatrix} \alpha \geq \beta \end{bmatrix}$$

The remaining branches are pruned (ignored) because they cannot improve the final decision.

Example



Left subtree returns **3**.

So,

$$\text{Alpha} = 3$$

Now evaluating the right subtree,

MIN already finds 2.

Since

$\beta = 2$

and

$\alpha = 3$

Therefore,

[
 $\alpha \geq \beta$
]

The remaining branch is pruned because MAX will never choose it.

Advantages

- Evaluates fewer nodes.
- Produces the same optimal result as Minimax.
- Reduces computation time.
- Improves search speed.
- Suitable for real-time games.

Comparison

Minimax	Alpha-Beta Pruning
Searches all nodes	Removes unnecessary nodes
Slower	Faster
High computation	Reduced computation
Same optimal result	Same optimal result

iv. Design an Evaluation Function and Justify the Factors Considered

When the search cannot reach the end of the game, the AI estimates the quality of a game state using an evaluation function.

Example Evaluation Function

$$\begin{aligned} &[\\ \text{Score} &= 5R + 4A + 3T + 2D - 4E \\ &] \end{aligned}$$

Where:

- **R** = Available Resources
- **A** = Army Strength
- **T** = Territory Controlled
- **D** = Defensive Position
- **E** = Enemy Strength

Factors Considered

1. Resources

More resources allow stronger units and better strategies.

2. Army Strength

A larger and healthier army increases the chance of winning.

3. Territory Control

Controlling important areas provides strategic advantages.

4. Defensive Position

Strong defenses reduce losses during enemy attacks.

5. Enemy Strength

Higher enemy strength decreases the overall evaluation score.

Example

Suppose:

Resources = 10

Army = 8

Territory = 6

Defense = 5

Enemy Strength = 7

Then,

[
Score = $5(10)+4(8)+3(6)+2(5)-4(7)$
]

[
= $50+32+18+10-28$
]

[
= 82
]

A higher score indicates a better game position.

v. Recommend a Strategy for Balancing Optimality and Real-Time Performance

Recommended Strategy

Minimax + Alpha-Beta Pruning + Depth Limit + Evaluation Function

This combination provides the best balance between decision quality and speed.

Justification

- Minimax selects the optimal move assuming an intelligent opponent.
- Alpha-Beta Pruning eliminates unnecessary branches, reducing computation.
- Depth Limit restricts the search to a manageable number of levels, allowing decisions within strict time limits.
- Evaluation Function estimates the quality of non-terminal game states, enabling effective decisions even without exploring the full game tree.

This combination ensures that the AI responds quickly while maintaining strong gameplay.

Final Comparison

Feature	Minimax	Alpha-Beta Pruning
Optimal Decision	Yes	Yes
Searches All Nodes	Yes	No
Speed	Slow	Fast
Memory Usage	High	Lower
Suitable for Real-Time Games	Moderate	Excellent